

Embeddings

DS 4440 · LECTURE 8

1. Today

- Our first foray into **representation learning**
- How to encode text
- Word2Vec (and beyond)

The goal is to assign every word a short vector of numbers, so that words used in similar ways get similar vectors.

$$\begin{array}{l} \begin{bmatrix} -0.4 & 1.2 & 0.7 \end{bmatrix} \\ \text{"cat"} \\ \\ \begin{bmatrix} -0.3 & 1.1 & 0.9 \end{bmatrix} \\ \text{"dog"} \end{array} \qquad \begin{array}{l} \begin{bmatrix} 2.6 & -1.8 & 0.2 \end{bmatrix} \\ \text{"pancake"} \end{array}$$

2. Way back (~2013): bag-of-words

A document becomes a vector as long as the vocabulary. Write w_{idx} for the position assigned to word w :

$$x_{w_{\text{idx}}} = \begin{cases} 1 & \text{if } w \text{ is in the input,} \\ 0 & \text{otherwise.} \end{cases}$$

$$\begin{array}{c} \text{This} \\ \text{movie} \\ \text{is} \\ \text{great} \\ \text{!} \end{array} \xrightarrow{\phi} \begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix} \begin{array}{l} \text{a} \\ \text{aardvark} \\ \vdots \\ \text{great} \\ \vdots \\ \text{mop} \\ \text{movie} \\ \text{mow} \\ \vdots \\ \text{zany} \\ \text{zebra} \end{array} \end{array} \left. \vphantom{\begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix} \end{array}} \right\} 50\text{k}$$

3. Problems with bag-of-words

The vectors are long and sparse. The deeper problem shows up when we try to measure text similarity:

$$\cos(a, b) = \frac{\phi(a)^\top \phi(b)}{\|\phi(a)\| \|\phi(b)\|}.$$

For “cat” and “dog”, the two vectors have their 1s in different positions, so the dot product is zero:

$$[0 \dots \underbrace{1}_{\text{cat}} \dots 0] \cdot [0 \dots \underbrace{1}_{\text{dog}} \dots 0] = 0.$$

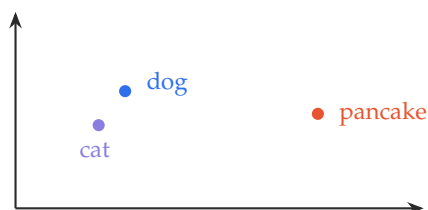
$$\text{sim}(\text{cat}, \text{dog}) = \text{sim}(\text{cat}, \text{pancake}) = 0$$

Every pair of distinct words is equally dissimilar. The representation knows nothing about meaning because we assigned the positions arbitrarily.

Distributional semantics

“You shall know a word by the company it keeps.”

If two words occur in the same kinds of contexts, treat them as similar. So instead of one position per word, we want **low-dimensional embeddings** in which similar words land near each other.



4. Word2Vec [Mikolov 2013]

Pick a **target** word and the **context** words around it. Two variants, differing in which one predicts the other.

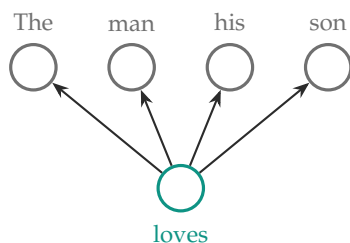
Two matrices, not one

Each word gets *two* vectors: v_w for when it acts as the target, and u_w for when it acts as context. So there are two embedding matrices,

$$E^{\text{in}}, E^{\text{out}} \in \mathbb{R}^{|V| \times d},$$

and a (target, context) pair is scored by $v_{w_t} \cdot u_{w_c}$. Keeping the roles separate matters: with a single matrix the score of a word with itself, $v_w \cdot v_w = \|v_w\|^2$, is as large as the score can get, and the objective would push every word to predict itself. At the end we keep E^{in} and throw E^{out} away.

Skip-gram: target predicts context



$$P(\text{The} \mid \text{loves}) \cdot P(\text{man} \mid \text{loves}) \cdot \\ P(\text{his} \mid \text{loves}) \cdot P(\text{son} \mid \text{loves})$$

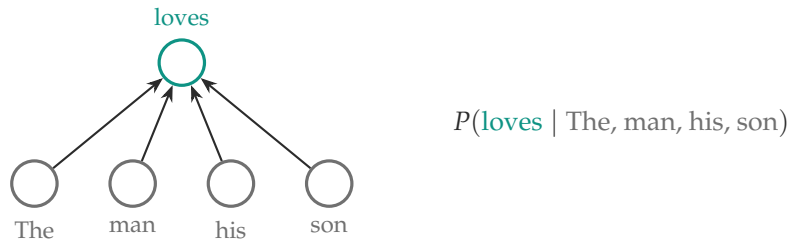
Each conditional is a softmax over the whole vocabulary:

$$P(w_c | w_t) \stackrel{\text{def}}{=} \frac{\exp\{v_{w_t} \cdot u_{w_c}\}}{\sum_{\tilde{w} \in V} \exp\{v_{w_t} \cdot u_{\tilde{w}}\}}$$

and the loss sums over every target in the corpus and every context word around it. We sum logs rather than multiplying probabilities, since a product of billions of probabilities underflows:

$$\mathcal{L} = - \sum_{w_t} \sum_{w_c \in \text{context}(w_t)} \log P(w_c | w_t).$$

CBOW: context predicts target



Average the context vectors first, then score the target against that average:

$$\bar{v}_c \stackrel{\text{def}}{=} \frac{1}{k} \sum_{j=1}^k v_{w_{c_j}}, \quad P(w_t | w_{c_1} \dots w_{c_k}) = \frac{\exp\{u_{w_t} \cdot \bar{v}_c\}}{\sum_{\tilde{w} \in V} \exp\{u_{\tilde{w}} \cdot \bar{v}_c\}},$$

with loss $-\log P(w_t | w_{c_1} \dots w_{c_k})$ summed over positions.

5. V is big, so this is slow

Both losses have a denominator that sums over the entire vocabulary. Every single update touches all 50k output vectors, which makes training impractical.

Negative sampling

Change the question. Instead of “which of the 50k words comes next,” ask “is this (target, context) pair real, or did I make it up?” That is binary classification, and we already know how to do it: one positive pair labelled 1, k sampled pairs labelled 0, and the BCE loss from Lecture 3.

$$\mathcal{L} = \underbrace{-\log \sigma(v_{w_t} \cdot u_{w_c})}_{\text{the real pair: push up}} - \underbrace{\sum_{j=1}^k \log \sigma(-v_{w_t} \cdot u_{\tilde{w}_j})}_{k \text{ sampled pairs: push down}}$$

The minus sign inside the second σ is doing the work: since $\sigma(-x) = 1 - \sigma(x)$, the second term is the usual $-\log(1 - \hat{y})$ for a negative example. There is no sum over V anywhere, because there is no longer a distribution over V to normalize — only $k + 1$ dot products per update, with k around 5 to 20.

Negatives are drawn from a unigram distribution raised to the $\frac{3}{4}$ power,

$$P_n(w) \propto \text{count}(w)^{3/4},$$

which samples rare words more often than their frequency alone would. The exponent is empirical.

One update, by hand

Take $V = \{\text{the, cat, sat, on, mat, pancake}\}$, $d = 2$, and the sentence “the cat sat on the mat” with a window of 1. For the target **cat** the real context word is sat; sample **pancake** and **on** as negatives. Say the current vectors are

$$\mathbf{v}_{w_t} = [0.5 \quad -0.2], \quad \mathbf{u}_{\text{sat}} = [0.8 \quad 0.4], \quad \mathbf{u}_{\text{pancake}} = [-0.6 \quad 0.7], \quad \mathbf{u}_{\text{on}} = [0.1 \quad 0.9].$$

Forward. Three dot products, three logistic terms:

pair	label y	$\mathbf{v}_{w_t} \cdot \mathbf{u}$	$\hat{y} = \sigma(\mathbf{v}_{w_t} \cdot \mathbf{u})$	loss term	$\hat{y} - y$
(cat, sat)	1	0.32	0.579	$-\log \hat{y} = 0.546$	-0.421
(cat, pancake)	0	-0.44	0.392	$-\log(1 - \hat{y}) = 0.497$	0.392
(cat, on)	0	-0.13	0.468	$-\log(1 - \hat{y}) = 0.630$	0.468
$\mathcal{L} = 1.673$					

$\hat{y}$ means the same thing in every row: the probability the model assigns to the pair being real. For the negatives we want $1 - \hat{y}$, and rather than computing it that way we use $\sigma(-x) = 1 - \sigma(x)$ — so $1 - 0.392 = \sigma(0.44) = 0.608$, and the loss term is $-\log \sigma(-\mathbf{v}_{w_t} \cdot \mathbf{u})$ as written in the box above.

Backward. Differentiating a term with respect to its dot product $z = \mathbf{v}_{w_t} \cdot \mathbf{u}$ gives $\hat{y} - y$ — the same (prediction - target) from Lectures 5 and 6, since this is BCE again. That is the last column of the table.

Worth doing one negative explicitly, since the sign flip makes it look like a different calculation. For **pancake** the loss term is $-\log \sigma(-z)$, and $\frac{d}{dz} \log \sigma(-z) = -\sigma(z)$, so

$$\frac{\partial \mathcal{L}}{\partial z} = \sigma(z) = \sigma(-0.44) = 0.392 = \hat{y} - 0.$$

Note it is $\hat{y} = 0.392$ that appears here, not the 0.608 from the loss column. Then $z = \mathbf{v}_{w_t} \cdot \mathbf{u}_{\text{pancake}}$ is a dot product, so $\partial z / \partial \mathbf{u}_{\text{pancake}} = \mathbf{v}_{w_t}$ and

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_{\text{pancake}}} = 0.392 \mathbf{v}_{w_t} = 0.392 [0.5 \quad -0.2] = [0.196 \quad -0.078].$$

The sign tells us what happens next: the gradient points along $+\mathbf{v}_{w_t}$, so the step $-\alpha(0.392)\mathbf{v}_{w_t}$ moves $\mathbf{u}_{\text{pancake}}$ away from $\mathbf{v}_{w_t}$ and the dot product goes down.

The other context vectors work the same way, each scaled by its own $\hat{y} - y$. The target’s gradient collects all three pairs:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_{\text{sat}}} = -0.421 \mathbf{v}_{w_t} = [-0.210 \quad 0.084], \quad \frac{\partial \mathcal{L}}{\partial \mathbf{v}_{w_t}} = \sum_{\text{pairs}} (\hat{y} - y) \mathbf{u} = [-0.525 \quad 0.527].$$

One step, $\alpha = 0.1$:

$$\mathbf{v}_{w_t} : [0.5 \quad -0.2] \rightarrow [0.552 \quad -0.253], \quad \mathbf{u}_{\text{sat}} : [0.8 \quad 0.4] \rightarrow [0.821 \quad 0.392],$$

$$\mathbf{u}_{\text{pancake}} : [-0.6 \quad 0.7] \rightarrow [-0.620 \quad 0.708], \quad \mathbf{u}_{\text{on}} : [0.1 \quad 0.9] \rightarrow [0.077 \quad 0.909].$$

Check that it moved the right way: $\mathbf{v}_{w_t} \cdot \mathbf{u}_{\text{sat}}$ goes from 0.32 to 0.355, and $\mathbf{v}_{w_t} \cdot \mathbf{u}_{\text{pancake}}$ from -0.44 to -0.521. Cat and sat got closer; cat and pancake got further apart.

Note what did *not* happen: mat and the were never touched. Four rows across the two matrices received gradient, out of twelve here and out of 100k in a real run. That is the whole saving.

In words, for each variant

	push together	push apart
skip-gram	w_t and each real context w_{c_ℓ}	w_t and words from other contexts
CBOW	w_t and the context average $\bar{v}_c$	random $\tilde{w}$ and $\bar{v}_c$

6. What an embedding layer actually is

Nothing new. Let $x_w \in \mathbb{R}^{(1 \times |V|)}$ be the one-hot row for word w and $E \in \mathbb{R}^{(|V| \times d)}$ the embedding matrix. Then

$$e_w = x_w E$$

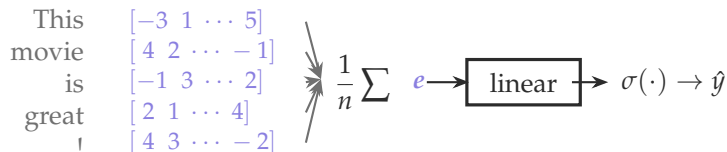
is a linear layer, and because x_w is one-hot the product just selects row w of E . Implementations skip the multiply and index the row directly, which is why it is called a lookup table. The gradient reaches only the rows that appeared in the batch.

This also explains why similar words end up with similar vectors. Two words that occur in the same contexts have to produce the same predictions, and the only way to do that through a dot product is to have nearly equal rows.

On size: with $|V| = 50k$ and $d = 300$, each matrix holds 15M parameters. The embedding table is usually far larger than whatever model sits on top of it.

7. Using embeddings for text classification

One way: adopt CBOW. Look up an embedding per word, average them, and put a classifier on top.



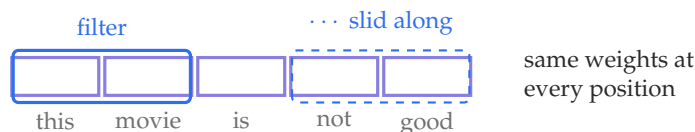
One choice to be explicit about: whether the gradient is allowed to reach E . **Fine-tuning** the embeddings usually helps when the labelled set is large, and overfits when it is small, in which case we **freeze** them.

Averaging throws away order

The classifier above averages the word vectors, so

$$\phi(\text{"not good"}) = \phi(\text{"good not"}).$$

Same words, same representation, and no classifier on top can tell them apart. The embeddings are fine; the averaging is what loses the information. What we want is to look at *adjacent* words — “not” followed by “good” is a pattern worth detecting, wherever in the sentence it appears:



A set of weights applied to a window of adjacent embeddings, reused at every position, is a **convolution**. That is where we pick up next lecture.

8. Beyond words

Embeddings are not just for NLP. Anything that comes in sequences, or can be made to, can be fed to the same objective. **DeepWalk** (Perozzi et al.) samples random walks over a graph and treats each walk as a “sentence.” Back in NLP, a document can be treated as one more “word” sitting in the context of every token it contains, which gives it a vector of its own (doc2vec).



9. Static embeddings

One vector per word *type*, so “bank” gets a single vector covering the river and the financial institution, and “not” gets one vector regardless of what it negates. We come back to that later in the course.

(See the Colab notebook.)